

Spark Security

- [Spark Security: Things You Need To Know \(#spark-security-things-you-need-to-know\)](#)
- [Spark RPC \(Communication protocol between Spark processes\) \(#spark-rpc-communication-protocol-between-spark-processes\)](#)
 - [Authentication \(#authentication\)](#)
 - [YARN \(#yarn\)](#)
 - [Kubernetes \(#kubernetes\)](#)
- [Network Encryption \(#network-encryption\)](#)
 - [SSL Encryption \(Preferred\) \(#ssl-encryption-preferred\)](#)
 - [AES-based Encryption \(Legacy\) \(#aes-based-encryption-legacy\)](#)
- [Local Storage Encryption \(#local-storage-encryption\)](#)
- [Web UI \(#web-ui\)](#)
 - [Authentication and Authorization \(#authentication-and-authorization\)](#)
 - [Spark History Server ACLs \(#spark-history-server-acls\)](#)
 - [SSL Configuration \(#ssl-configuration\)](#)
 - [Preparing the key stores \(#preparing-the-key-stores\)](#)
 - [YARN mode \(#yarn-mode\)](#)
 - [Standalone mode \(#standalone-mode\)](#)
 - [HTTP Security Headers \(#http-security-headers\)](#)
- [Configuring Ports for Network Security \(#configuring-ports-for-network-security\)](#)
 - [Standalone mode only \(#standalone-mode-only\)](#)
 - [All cluster managers \(#all-cluster-managers\)](#)
- [Kerberos \(#kerberos\)](#)
 - [Long-Running Applications \(#long-running-applications\)](#)
 - [Using a Keytab \(#using-a-keytab\)](#)
 - [Using a ticket cache \(#using-a-ticket-cache\)](#)
 - [Secure Interaction with Kubernetes \(#secure-interaction-with-kubernetes\)](#)
- [Event Logging \(#event-logging\)](#)
- [Persisting driver logs in client mode \(#persisting-driver-logs-in-client-mode\)](#)

Spark Security: Things You Need To Know

Security features like authentication are not enabled by default. When deploying a cluster that is open to the internet or an untrusted network, it's important to secure access to the cluster to prevent unauthorized applications from running on the cluster.

Spark supports multiple deployments types and each one supports different levels of security. Not all deployment types will be secure in all environments and none are secure by default. Be sure to evaluate your environment, what Spark supports, and take the appropriate measure to secure your Spark deployment.

There are many different types of security concerns. Spark does not necessarily protect against all things. Listed below are some of the things Spark supports. Also check the deployment documentation for the type of deployment you are using for deployment specific settings. Anything not documented, Spark does not support.

Spark RPC (Communication protocol between Spark processes)

Authentication

Spark currently supports authentication for RPC channels using a shared secret. Authentication can be turned on by setting the `spark.authenticate` configuration parameter.

The exact mechanism used to generate and distribute the shared secret is deployment-specific. Unless specified below, the secret must be defined by setting the `spark.authenticate.secret` config option. The same secret is shared by all Spark applications and daemons in that case, which limits the security of these deployments, especially on multi-tenant clusters.

The REST Submission Server supports HTTP Authorization header with a cryptographically signed JSON Web Token via `JWSFilter`. To enable authorization, Spark Master should have `spark.master.rest.filters=org.apache.spark.ui.JWSFilter` and `spark.org.apache.spark.ui.JWSFilter.param.secretKey=BASE64URL-ENCODED-KEY` configurations, and client should provide HTTP Authorization header which contains JSON Web Token signed by the shared secret key.

YARN

For Spark on [YARN \(running-on-yarn.html\)](#), Spark will automatically handle generating and distributing the shared secret. Each application will use a unique shared secret. In the case of YARN, this feature relies on YARN RPC encryption being enabled for the distribution of secrets to be secure.

Property Name	Default	Meaning	Since Version
<code>spark.yarn.shuffle.server.recovery.disabled</code>	false	Set to true for applications that have higher security requirements and prefer that their secret is not saved in the db. The shuffle data of such applications will not be recovered after the External Shuffle Service restarts.	3.5.0

Kubernetes

On Kubernetes, Spark will also automatically generate an authentication secret unique to each application. The secret is propagated to executor pods using environment variables. This means that any user that can list pods in the namespace where the Spark application is running can also see their authentication secret. Access control rules should be properly set up by the Kubernetes admin to ensure that Spark authentication is secure.

Property Name	Default	Meaning	Since Version
<code>spark.authenticate</code>	false	Whether Spark authenticates its internal connections.	1.0.0
<code>spark.authenticate.secret</code>	None	The secret key used authentication. See above for when this configuration should be set.	1.0.0

Alternatively, one can mount authentication secrets using files and Kubernetes secrets that the user mounts into their pods.

Property Name	Default	Meaning	Since Version
<code>spark.authenticate.secret.file</code>	None	Path pointing to the secret key to use for securing connections. Ensure that the contents of the file have been securely generated. This file is loaded on both the driver and the executors unless other settings override this (see below).	3.0.0
<code>spark.authenticate.secret.driver.file</code>	The value of <code>spark.authenticate.secret.file</code>	When specified, overrides the location that the Spark driver reads to load the secret. Useful when in client mode, when the location of the secret file may differ in the pod versus the node the driver is running in. When this is specified, <code>spark.authenticate.secret.executor.file</code> must be specified so that the driver and the executors can both use files to load the secret key. Ensure that the contents of the file on the driver is identical to the contents of the file on the executors.	3.0.0
<code>spark.authenticate.secret.executor.file</code>	The value of <code>spark.authenticate.secret.file</code>	When specified, overrides the location that the Spark executors read to load the secret. Useful in client mode, when the location of the secret file may differ in the pod versus the node the driver is running in. When this is specified, <code>spark.authenticate.secret.driver.file</code> must be specified so that the driver and the executors can both use files to load the secret key. Ensure that the contents of the file on the driver is identical to the contents of the file on the executors.	3.0.0

Note that when using files, Spark will not mount these files into the containers for you. It is up you to ensure that the secret files are deployed securely into your containers and that the driver’s secret file agrees with the executors’ secret file.

Network Encryption

Spark supports two mutually exclusive forms of encryption for RPC connections:

The **preferred method** uses TLS (aka SSL) encryption via Netty’s support for SSL. Enabling SSL requires keys and certificates to be properly configured. SSL is standardized and considered more secure.

The legacy method is an AES-based encryption mechanism relying on a shared secret. This requires RPC authentication to also be enabled. This method uses a bespoke protocol and it is recommended to use SSL instead.

One may prefer to use the SSL based encryption in scenarios where compliance mandates the usage of specific protocols; or to leverage the security of a more standard encryption library. However, the AES based encryption is simpler to configure and may be preferred if the only requirement is that data be encrypted in transit.

If both options are enabled in the configuration, the SSL based RPC encryption takes precedence and the AES based encryption will not be used (and a warning message will be emitted).

SSL Encryption (Preferred)

Spark supports SSL based encryption for RPC connections. Please refer to the SSL Configuration section below to understand how to configure it. The SSL settings are mostly similar across the UI and RPC, however there are a few additional settings which are specific to the RPC implementation. The RPC implementation uses Netty under the hood (while the UI uses Jetty), which supports a different set of options.

Unlike the other SSL settings for the UI, the RPC SSL is *not* automatically enabled if `spark.ssl.enabled` is set. It must be explicitly enabled, to ensure a safe migration path for users upgrading Spark versions.

AES-based Encryption (Legacy)

Spark supports AES-based encryption for RPC connections. For encryption to be enabled, RPC authentication must also be enabled and properly configured. AES encryption uses the [Apache Commons Crypto](https://commons.apache.org/proper/commons-crypto/) (<https://commons.apache.org/proper/commons-crypto/>) library, and Spark’s configuration system allows access to that library’s configuration for advanced users.

This legacy protocol has two mutually incompatible versions. Version 1 omits applying key derivation function (KDF) to the key exchange protocol’s output, while version 2 applies a KDF to ensure that the derived session key is uniformly distributed. Version 1 is default for backward compatibility. It is **recommended to use version 2** for better security properties. The version can be configured by setting `spark.network.crypto.authEngineVersion` to 1 or 2 respectively.

There is also support for SASL-based encryption, although it should be considered deprecated. It is still required when talking to shuffle services from Spark versions older than 2.2.0.

The following table describes the different options available for configuring this feature.

Property Name	Default	Meaning	Since Version
<code>spark.network.crypto.enabled</code>	false	Enable AES-based RPC encryption, including the new authentication protocol added in 2.2.0.	2.2.0
<code>spark.network.crypto.cipher</code>	AES/CTR/NoPadding	Cipher mode to use. Defaults "AES/CTR/NoPadding" for backward compatibility, which is not authenticated. Recommended to use "AES/GCM/NoPadding", which is an authenticated encryption mode.	4.0.0, 3.5.2, 3.4.4
<code>spark.network.crypto.authEngineVersion</code>	1	Version of AES-based RPC encryption to use. Valid versions are 1 or 2. Version 2 is recommended.	4.0.0
<code>spark.network.crypto.config.*</code>	None	Configuration values for the commons-crypto library, such as which cipher implementations to use. The config name should be the name of commons-crypto configuration without the <code>commons.crypto</code> prefix.	2.2.0
<code>spark.network.crypto.saslFallback</code>	true	Whether to fall back to SASL authentication if authentication fails using Spark's internal mechanism. This is useful when the application is connecting to old shuffle services that do not support the internal Spark authentication protocol. On the shuffle service side, disabling this feature will block older clients from authenticating.	2.2.0
<code>spark.authenticate.enableSaslEncryption</code>	false	Enable SASL-based encrypted communication.	2.2.0
<code>spark.network.sasl.serverAlwaysEncrypt</code>	false	Disable unencrypted connections for ports using SASL authentication. This will deny connections from clients that have authentication enabled, but do not request SASL-based encryption.	1.4.0

Local Storage Encryption

Spark supports encrypting temporary data written to local disks. This covers shuffle files, shuffle spills and data blocks stored on disk (for both caching and broadcast variables). It does not cover encrypting output data generated by applications with APIs such as `saveAsHadoopFile` or `saveAsTable`. It also may not cover temporary files created explicitly by the user.

The following settings cover enabling encryption for data written to disk:

Property Name	Default	Meaning	Since Version
<code>spark.io.encryption.enabled</code>	false	Enable local disk I/O encryption. Currently supported by all modes. It's strongly recommended that RPC encryption be enabled when using this feature.	2.1.0
<code>spark.io.encryption.keySizeBits</code>	128	IO encryption key size in bits. Supported values are 128, 192 and 256.	2.1.0
<code>spark.io.encryption.keygen.algorithm</code>	HmacSHA1	The algorithm to use when generating the IO encryption key. The supported algorithms are described in the KeyGenerator section of the Java Cryptography Architecture Standard Algorithm Name Documentation.	2.1.0
<code>spark.io.encryption.common.config.*</code>	None	Configuration values for the commons-crypto library, such as which cipher implementations to use. The config name should be the name of commons-crypto configuration without the <code>commons.crypto</code> prefix.	2.1.0

Web UI

Authentication and Authorization

Enabling authentication for the Web UIs is done using [jakarta servlet filters](https://jakarta.ee/specifications/servlet/5.0/apidocs/jakarta/servlet/filter) (<https://jakarta.ee/specifications/servlet/5.0/apidocs/jakarta/servlet/filter>). You will need a filter that implements the authentication method you want to deploy. Spark does not provide any built-in authentication filters.

Spark also supports access control to the UI when an authentication filter is present. Each application can be configured with its own separate access control lists (ACLs). Spark differentiates between “view” permissions (who is allowed to see the application’s UI), and “modify” permissions (who can do things like kill jobs in a running application).

ACLs can be configured for either users or groups. Configuration entries accept comma-separated lists as input, meaning multiple users or groups can be given the desired privileges. This can be used if you run on a shared cluster and have a set of administrators or developers who need to monitor applications they may not have started themselves. A wildcard (*) added to specific ACL means that all users will have the respective privilege. By default, only the user submitting the application is added to the ACLs.

Group membership is established by using a configurable group mapping provider. The mapper is configured using the `spark.user.groups.mapping` config option, described in the table below.

The following options control the authentication of Web UIs:

Property Name	Default	Meaning	Since Version
<code>spark.ui.allowFramingFrom</code>	SAMEORIGIN	Allow framing for a specific named URI via <code>X-Frame-Options</code> . By default, allow only from the same origin.	1.6.0
<code>spark.ui.filters</code>	None	Spark supports HTTP Authorization header with a cryptographically signed JSON Web Token via <code>org.apache.spark.ui.JWSFilter</code> . See the Spark UI (configuration.html#spark-ui) configuration for how to configure filters.	1.0.0
<code>spark.acls.enable</code>	false	Whether UI ACLs should be enabled. If enabled, this checks to see if the user has access permissions to view or modify the application. Note this requires the user to be authenticated, so if no authentication filter is installed, this option does not do anything.	1.1.0
<code>spark.admin.acls</code>	None	Comma-separated list of users that have view and modify access to the Spark application.	1.1.0

Property Name	Default	Meaning	Since Version
spark.admin.acls.groups	None	Comma-separated list of groups that have view and modify access to the Spark application.	2.0.0
spark.modify.acls	None	Comma-separated list of users that have modify access to the Spark application.	1.1.0
spark.modify.acls.groups	None	Comma-separated list of groups that have modify access to the Spark application.	2.0.0
spark.ui.view.acls	None	Comma-separated list of users that have view access to the Spark application.	1.0.0
spark.ui.view.acls.groups	None	Comma-separated list of groups that have view access to the Spark application.	2.0.0
spark.user.groups.mapping	org.apache.spark.security.ShellBasedGroupsMappingProvider	The list of groups for a user is determined by a group mapping service defined by the trait <code>org.apache.spark.security.GroupMappingService</code> <code>ServiceProvider</code>, which can be configured by this property. By default, a Unix shell-based implementation is used, which collects this information from the host OS. <i>Note:</i> This implementation supports only Unix/Linux-based environments. Windows environment is currently not supported. However, a new platform/protocol can be supported by implementing the trait mentioned above.	2.0.0

On YARN, the view and modify ACLs are provided to the YARN service when submitting applications, and control who has the respective privileges via YARN interfaces.

Spark History Server ACLs

Authentication for the SHS Web UI is enabled the same way as for regular applications, using servlet filters.

To enable authorization in the SHS, a few extra options are used:

Property Name	Default	Meaning	Since Version
spark.history.ui.acls.enable	false	Specifies whether ACLs should be checked to authorize users viewing the applications in the history server. If enabled, access control checks are performed regardless of what the individual applications had set for <code>spark.ui.acls.enable</code> . The application owner will always have authorization to view their own application and any users specified via <code>spark.ui.view.acls</code> and groups specified via <code>spark.ui.view.acls.groups</code> when the application was run will also have authorization to view that application. If disabled, no access control checks are made for any application UIs available through the history server.	1.0.1
spark.history.ui.admin.acls	None	Comma separated list of users that have view access to all the Spark applications in history server.	2.1.1
spark.history.ui.admin.acls.groups	None	Comma separated list of groups that have view access to all the Spark applications in history server.	2.1.1

The SHS uses the same options to configure the group mapping provider as regular applications. In this case, the group mapping provider will apply to all UIs server by the SHS, and individual application configurations will be ignored.

SSL Configuration

Configuration for SSL is organized hierarchically. The user can configure the default SSL settings which will be used for all the supported communication protocols unless they are overwritten by protocol-specific settings. This way the user can easily provide the common settings for all the protocols without disabling the ability to configure each one individually. Note that all

settings are inherited this way, *except* for `spark.ssl.rpc.enabled` which must be explicitly set.

The following table describes the SSL configuration namespaces:

Config Namespace	Component
<code>spark.ssl</code>	The default SSL configuration. These values will apply to all namespaces below, unless explicitly overridden at the namespace level.
<code>spark.ssl.ui</code>	Spark application Web UI
<code>spark.ssl.standalone</code>	Standalone Master / Worker Web UI
<code>spark.ssl.historyServer</code>	History Server Web UI
<code>spark.ssl.rpc</code>	Spark RPC communication

The full breakdown of available SSL options can be found below. The `${ns}` placeholder should be replaced with one of the above namespaces.

Property Name	Default	Meaning	Supported Namespaces
<code>\${ns}.enabled</code>	false	Enables SSL. When enabled, <code>\${ns}.ssl.protocol</code> is required.	ui,standalone,historyServer
<code>\${ns}.port</code>	None	The port where the SSL service will listen on. The port must be defined within a specific namespace configuration. The default namespace is ignored when reading this configuration. When not set, the SSL port will be derived from the non-SSL port for the same service. A value of "0" will make the service bind to an ephemeral port.	ui,standalone,historyServer
<code>\${ns}.enabledAlgorithms</code>	None	A comma-separated list of ciphers. The specified ciphers must be supported by JVM. The reference list of protocols can be found in the "JSSE Cipher Suite Names" section of the Java security guide. The list for Java 17 can be found at this (https://docs.oracle.com/en/java/javase/17/docs/specs/security/standard-names.html#jsse-cipher-suite-names) page. Note: If not set, the default cipher suite for the JRE will be used.	ui,standalone,historyServer
<code>\${ns}.keyPassword</code>	None	The password to the private key in the key store.	ui,standalone,historyServer
<code>\${ns}.keyStore</code>	None	Path to the key store file. The path can be absolute or relative to the directory in which the process is started.	ui,standalone,historyServer
<code>\${ns}.keyStorePassword</code>	None	Password to the key store.	ui,standalone,historyServer
<code>\${ns}.keyStoreType</code>	JKS	The type of the key store.	ui,standalone,historyServer
<code>\${ns}.protocol</code>	None	TLS protocol to use. The protocol must be supported by JVM. The reference list of protocols can be found in the "Additional JSSE Standard Names" section of the Java security guide. For Java 17, the list can be found at this (https://docs.oracle.com/en/java/javase/17/docs/specs/security/standard-names.html#additional-jsse-standard-names) page.	ui,standalone,historyServer
<code>\${ns}.needClientAuth</code>	false	Whether to require client authentication.	ui,standalone,historyServer
<code>\${ns}.trustStore</code>	None	Path to the trust store file. The path can be absolute or relative to the directory in which the process is started.	ui,standalone,historyServer
<code>\${ns}.trustStorePassword</code>	None	Password for the trust store.	ui,standalone,historyServer
<code>\${ns}.trustStoreType</code>	JKS	The type of the trust store.	ui,standalone,historyServer
<code>\${ns}.opensslEnabled</code>	false	Whether to use OpenSSL for cryptographic operations instead of the JDK SSL provider. This setting requires the <code>`certChain`</code> and <code>`privateKey`</code> settings to be set. This takes precedence over the <code>`keyStore`</code> and <code>`trustStore`</code> settings if both are specified. If the OpenSSL library is not available at runtime, we will fall back to the JDK provider.	rpc
<code>\${ns}.privateKey</code>	None	Path to the private key file in PEM format. The path can be absolute or relative to the directory in which the process is started. This setting is required when using the OpenSSL implementation.	rpc
<code>\${ns}.privateKeyPassword</code>	None	The password to the above private key file in PEM format.	rpc
<code>\${ns}.certChain</code>	None	Path to the certificate chain file in PEM format. The path can be absolute or relative to the directory in which the process is started. This setting is required when using the OpenSSL implementation.	rpc

Property Name	Default	Meaning	Supported Namespaces
<code>\${ns}.trustStoreReloadingEnabled</code>	false	Whether the trust store should be reloaded periodically. This setting is mostly only useful in standalone deployments, not k8s or yarn deployments.	rpc
<code>\${ns}.trustStoreReloadIntervalMs</code>	10000	The interval at which the trust store should be reloaded (in milliseconds). This setting is mostly only useful in standalone deployments, not k8s or yarn deployments.	rpc

Spark also supports retrieving `${ns}.keyPassword`, `${ns}.keyStorePassword` and `${ns}.trustStorePassword` from [Hadoop Credential Providers \(https://hadoop.apache.org/docs/current/hadoop-project-dist/hadoop-common/CredentialProviderAPI.html\)](https://hadoop.apache.org/docs/current/hadoop-project-dist/hadoop-common/CredentialProviderAPI.html). User could store password into credential file and make it accessible by different components, like:

```
hadoop credential create spark.ssl.keyPassword -value password \
    -provider jceks://hdfs@nn1.example.com:9001/user/backup/ssl.jceks
```

To configure the location of the credential provider, set the `hadoop.security.credential.provider.path` config option in the Hadoop configuration used by Spark, like:

```
<property>
  <name>hadoop.security.credential.provider.path</name>
  <value>jceks://hdfs@nn1.example.com:9001/user/backup/ssl.jceks</value>
</property>
```

Or via SparkConf
“spark.hadoop.hadoop.security.credential.provider.path=jceks://hdfs@nn1.example.com:9001/user/backup/ssl.jceks”.

Preparing the key stores

Key stores can be generated by `keytool` program. The reference documentation for this tool for Java 17 is [here \(https://docs.oracle.com/en/java/javase/17/docs/specs/man/keytool.html\)](https://docs.oracle.com/en/java/javase/17/docs/specs/man/keytool.html). The most basic steps to configure the key stores and the trust store for a Spark Standalone deployment mode is as follows:

- Generate a key pair for each node
- Export the public key of the key pair to a file on each node
- Import all exported public keys into a single trust store
- Distribute the trust store to the cluster nodes

YARN mode

To provide a local trust store or key store file to drivers running in cluster mode, they can be distributed with the application using the `--files` command line argument (or the equivalent `spark.files` configuration). The files will be placed on the driver’s working directory, so the TLS configuration should just reference the file name with no absolute path.

Distributing local key stores this way may require the files to be staged in HDFS (or other similar distributed file system used by the cluster), so it’s recommended that the underlying file system be configured with security in mind (e.g. by enabling authentication and wire encryption).

Standalone mode

The user needs to provide key stores and configuration options for master and workers. They have to be set by attaching appropriate Java system properties in `SPARK_MASTER_OPTS` and in `SPARK_WORKER_OPTS` environment variables, or just in `SPARK_DAEMON_JAVA_OPTS`.

The user may allow the executors to use the SSL settings inherited from the worker process. That can be accomplished by setting `spark.ssl.useNodeLocalConf` to `true`. In that case, the settings provided by the user on the client side are not used.

HTTP Security Headers

Apache Spark can be configured to include HTTP headers to aid in preventing Cross Site Scripting (XSS), Cross-Frame Scripting (XFS), MIME-Sniffing, and also to enforce HTTP Strict Transport Security.

Property Name	Default	Meaning	Since Version
<code>spark.ui.xsProtection</code>	<code>1; mode=block</code>	Value for HTTP X-XSS-Protection response header. You can choose appropriate value from below: • <code>0</code> (Disables XSS filtering) • <code>1</code> (Enables XSS filtering. If a cross-site scripting attack is detected, the browser will sanitize the page.) • <code>1; mode=block</code> (Enables XSS filtering. The browser will prevent rendering of the page if an attack is detected.)	2.3.0

Property Name	Default	Meaning	Since Version
<code>spark.ui.xContentTypeOptions.enabled</code>	<code>true</code>	When enabled, X-Content-Type-Options HTTP response header will be set to "nosniff".	2.3.0
<code>spark.ui.strictTransportSecurity</code>	None	Value for HTTP Strict Transport Security (HSTS) Response Header. You can choose appropriate value from below and set <code>expire-time</code> accordingly. This option is only used when SSL/TLS is enabled. <code>max-age=<expire-time></code> <code>max-age=<expire-time>; includeSubDomains</code> <code>max-age=<expire-time>; preload</code>	2.3.0

Configuring Ports for Network Security

Generally speaking, a Spark cluster and its services are not deployed on the public internet. They are generally private services, and should only be accessible within the network of the organization that deploys Spark. Access to the hosts and ports used by Spark services should be limited to origin hosts that need to access the services.

However, like the REST Submission port, Spark also supports HTTP Authorization header with a cryptographically signed JSON Web Token (JWT) for all UI ports. To use it, a user needs to configure `spark.ui.filters=org.apache.spark.ui.JWSFilter` and `spark.org.apache.spark.ui.JWSFilter.param.secretKey=BASE64URL-ENCODED-KEY`.

Below are the primary ports that Spark uses for its communication and how to configure those ports.

Standalone mode only

From	To	Default Port	Purpose	Configuration Setting	Notes
Browser	Standalone Master	8080	Web UI	<code>spark.master.ui.port</code> / <code>SPARK_MASTER_WEBUI_PORT</code>	Jetty-based. Standalone mode only.
Browser	Standalone Worker	8081	Web UI	<code>spark.worker.ui.port</code> / <code>SPARK_WORKER_WEBUI_PORT</code>	Jetty-based. Standalone mode only.
Driver / Standalone Worker	Standalone Master	7077	Submit job to cluster / Join cluster	<code>SPARK_MASTER_PORT</code>	Set to "0" to choose a port randomly. Standalone mode only.
External Service	Standalone Master	6066	Submit job to cluster via REST API	<code>spark.master.rest.port</code>	Use <code>spark.master.rest.enabled</code> to enable/disable this service. Standalone mode only.
Standalone Master	Standalone Worker	(random)	Schedule executors	<code>SPARK_WORKER_PORT</code>	Set to "0" to choose a port randomly. Standalone mode only.

All cluster managers

From	To	Default Port	Purpose	Configuration Setting	Notes
Browser	Application	4040	Web UI	<code>spark.ui.port</code>	Jetty-based
Browser	History Server	18080	Web UI	<code>spark.history.ui.port</code>	Jetty-based
Executor / Standalone Master	Driver	(random)	Connect to application / Notify executor state changes	<code>spark.driver.port</code>	Set to "0" to choose a port randomly.
Executor / Driver	Executor / Driver	(random)	Block Manager port	<code>spark.blockManager.port</code>	Raw socket via <code>ServerSocketChannel</code>

Kerberos

Spark supports submitting applications in environments that use Kerberos for authentication. In most cases, Spark relies on the credentials of the current logged in user when authenticating to Kerberos-aware services. Such credentials can be obtained by logging in to the configured KDC with tools like `kinit`.

When talking to Hadoop-based services, Spark needs to obtain delegation tokens so that non-local processes can authenticate. Spark ships with support for HDFS and other Hadoop file systems, Hive and HBase.

When using a Hadoop filesystem (such HDFS or WebHDFS), Spark will acquire the relevant tokens for the service hosting the user’s home directory.

An HBase token will be obtained if HBase is in the application’s classpath, and the HBase configuration has Kerberos authentication turned (`hbase.security.authentication=kerberos`).

Similarly, a Hive token will be obtained if Hive is in the classpath, and the configuration includes URIs for remote metastore services (`hive.metastore.uris` is not empty).

If an application needs to interact with other secure Hadoop filesystems, their URIs need to be explicitly provided to Spark at launch time. This is done by listing them in the `spark.kerberos.access.hadoopFileSystems` property, described in the configuration section below.

Spark also supports custom delegation token providers using the Java Services mechanism (see `java.util.ServiceLoader`). Implementations of `org.apache.spark.security.HadoopDelegationTokenProvider` can be made available to Spark by listing their names in the corresponding file in the jar’s `META-INF/services` directory.

Delegation token support is currently only supported in YARN and Kubernetes mode. Consult the deployment-specific page for more information.

The following options provides finer-grained control for this feature:

Property Name	Default	Meaning	Since Version
<code>spark.security.credentials.{service}.enabled</code>	<code>true</code>	Controls whether to obtain credentials for services when security is enabled. By default, credentials for all supported services are retrieved when those services are configured, but it's possible to disable that behavior if it somehow conflicts with the application being run.	2.3.0
<code>spark.kerberos.access.hadoopFileSystems</code>	(none)	A comma-separated list of secure Hadoop filesystems your Spark application is going to access. For example, <code>spark.kerberos.access.hadoopFileSystems=hdfs://nn1.com:8032,hdfs://nn2.com:8032,webhdfs://nn3.com:50070</code> . The Spark application must have access to the filesystems listed and Kerberos must be properly configured to be able to access them (either in the same realm or in a trusted realm). Spark acquires security tokens for each of the filesystems so that the Spark application can access those remote Hadoop filesystems.	3.0.0

Users can exclude Kerberos delegation token renewal at resource scheduler. Currently it is only supported on YARN. The configuration is covered in the [Running Spark on YARN \(running-on-yarn.html#yarn-specific-kerberos-configuration\)](#) page.

Long-Running Applications

Long-running applications may run into issues if their run time exceeds the maximum delegation token lifetime configured in services it needs to access.

This feature is not available everywhere. In particular, it’s only implemented on YARN and Kubernetes (both client and cluster modes).

Spark supports automatically creating new tokens for these applications. There are two ways to enable this functionality.

Using a Keytab

By providing Spark with a principal and keytab (e.g. using `spark-submit` with `--principal` and `--keytab` parameters), the application will maintain a valid Kerberos login that can be used to retrieve delegation tokens indefinitely.

Note that when using a keytab in cluster mode, it will be copied over to the machine running the Spark driver. In the case of YARN, this means using HDFS as a staging area for the keytab, so it’s strongly recommended that both YARN and HDFS be secured with encryption, at least.

Using a ticket cache

By setting `spark.kerberos.renewal.credentials` to `ccache` in Spark’s configuration, the local Kerberos ticket cache will be used for authentication. Spark will keep the ticket renewed during its renewable life, but after it expires a new ticket needs to be acquired (e.g. by running `kinit`).

It’s up to the user to maintain an updated ticket cache that Spark can use.

The location of the ticket cache can be customized by setting the `KRB5CCNAME` environment variable.

Secure Interaction with Kubernetes

When talking to Hadoop-based services behind Kerberos, it was noted that Spark needs to obtain delegation tokens so that non-local processes can authenticate. These delegation tokens in Kubernetes are stored in Secrets that are shared by the Driver and its Executors. As such, there are three ways of submitting a Kerberos job:

In all cases you must define the environment variable: `HADOOP_CONF_DIR` or `spark.kubernetes.hadoop.configMapName`.

It also important to note that the KDC needs to be visible from inside the containers.

If a user wishes to use a remote `HADOOP_CONF` directory, that contains the Hadoop configuration files, this could be achieved by setting `spark.kubernetes.hadoop.configMapName` to a pre-existing `ConfigMap`.

- 1. Submitting with a \$kinit that stores a TGT in the Local Ticket Cache:

```
/usr/bin/kinit -kt <keytab_file> <username>/<krb5 realm>
/opt/spark/bin/spark-submit \
  --deploy-mode cluster \
  --class org.apache.spark.examples.HdfsTest \
  --master k8s://<KUBERNETES_MASTER_ENDPOINT> \
  --conf spark.executor.instances=1 \
  --conf spark.app.name=spark-hdfs \
  --conf spark.kubernetes.container.image=spark:latest \
  --conf spark.kubernetes.kerberos.krb5.path=/etc/krb5.conf \
  local:///opt/spark/examples/jars/spark-examples_<VERSION>.jar \
  <HDFS_FILE_LOCATION>
```

- 2. Submitting with a local Keytab and Principal

```
/opt/spark/bin/spark-submit \
  --deploy-mode cluster \
  --class org.apache.spark.examples.HdfsTest \
  --master k8s://<KUBERNETES_MASTER_ENDPOINT> \
  --conf spark.executor.instances=1 \
  --conf spark.app.name=spark-hdfs \
  --conf spark.kubernetes.container.image=spark:latest \
  --conf spark.kerberos.keytab=<KEYTAB_FILE> \
  --conf spark.kerberos.principal=<PRINCIPAL> \
  --conf spark.kubernetes.kerberos.krb5.path=/etc/krb5.conf \
  local:///opt/spark/examples/jars/spark-examples_<VERSION>.jar \
  <HDFS_FILE_LOCATION>
```

- 3. Submitting with pre-populated secrets, that contain the Delegation Token, already existing within the namespace

```
/opt/spark/bin/spark-submit \
  --deploy-mode cluster \
  --class org.apache.spark.examples.HdfsTest \
  --master k8s://<KUBERNETES_MASTER_ENDPOINT> \
  --conf spark.executor.instances=1 \
  --conf spark.app.name=spark-hdfs \
  --conf spark.kubernetes.container.image=spark:latest \
  --conf spark.kubernetes.kerberos.tokenSecret.name=<SECRET_TOKEN_NAME> \
  --conf spark.kubernetes.kerberos.tokenSecret.itemKey=<SECRET_ITEM_KEY> \
  --conf spark.kubernetes.kerberos.krb5.path=/etc/krb5.conf \
  local:///opt/spark/examples/jars/spark-examples_<VERSION>.jar \
  <HDFS_FILE_LOCATION>
```

- 3b. Submitting like in (3) however specifying a pre-created krb5 ConfigMap and pre-created HADOOP_CONF_DIR ConfigMap

```
/opt/spark/bin/spark-submit \
  --deploy-mode cluster \
  --class org.apache.spark.examples.HdfsTest \
  --master k8s://<KUBERNETES_MASTER_ENDPOINT> \
  --conf spark.executor.instances=1 \
  --conf spark.app.name=spark-hdfs \
  --conf spark.kubernetes.container.image=spark:latest \
  --conf spark.kubernetes.kerberos.tokenSecret.name=<SECRET_TOKEN_NAME> \
  --conf spark.kubernetes.kerberos.tokenSecret.itemKey=<SECRET_ITEM_KEY> \
  --conf spark.kubernetes.hadoop.configMapName=<HCONF_CONFIG_MAP_NAME> \
  --conf spark.kubernetes.kerberos.krb5.configMapName=<KRB_CONFIG_MAP_NAME> \
  local:///opt/spark/examples/jars/spark-examples_<VERSION>.jar \
  <HDFS_FILE_LOCATION>
```

Event Logging

If your applications are using event logging, the directory where the event logs go (`spark.eventLog.dir`) should be manually created with proper permissions. To secure the log files, the directory permissions should be set to `drwxrwxrwt`. The owner and group of the directory should correspond to the super user who is running the Spark History Server.

This will allow all users to write to the directory but will prevent unprivileged users from reading, removing or renaming a file unless they own it. The event log files will be created by Spark with permissions such that only the user and group have read and write access.

Persisting driver logs in client mode

If your applications persist driver logs in client mode by enabling `spark.driver.log.persistToDfs.enabled`, the directory where the driver logs go (`spark.driver.log.dfsDir`) should be manually created with proper permissions. To secure the log files, the directory permissions should be set to `drwxrwxrwx`. The owner and group of the directory should correspond to the super user who is running the Spark History Server.

This will allow all users to write to the directory but will prevent unprivileged users from reading, removing or renaming a file unless they own it. The driver log files will be created by Spark with permissions such that only the user and group have read and write access.